Task 18 Unit test cases for microservices using a testing framework

Aim:

To develop and test a microservice using Node.js by writing unit test cases that validate service logic, REST API endpoints, and data handling, ensuring the reliability and correctness of the application.

Algorithm:

1. **Initialize the Server**

- Import required modules like Express.
- Create an Express application and enable JSON parsing.

2. **Create In-Memory Data**

- Define a sample dataset (e.g., student records) to simulate a database.

3. **Implement Service Logic**

- Write functions for:
    - Fetching all records
    - Fetching a record by ID
    - Adding a new record
    - Updating existing data
    - Deleting a record

4. **Develop REST API Endpoints**

- Create routes using HTTP methods:
    - GET → retrieve data
    - POST → add data
    - PUT → update data
    - DELETE → remove data

5. **Add Input Validation**

- Ensure required fields are provided in requests.
- Return proper error messages for invalid inputs.

6. **Write Unit Test Cases**

- Test each service function independently.
- Validate outputs against expected results.

- Check edge cases (invalid ID, missing data).

7. **Execute Tests**

- Run test functions automatically when the server starts.
- Display PASS/FAIL results in the console.

8. **Run the Application**

- Start the server using node server.js.
- Test endpoints using browser or Postman.

Code:

Server.js

```
const express = require("express");
const app = express();
app.use(express.json());

/* ========================
   HOME ROUTE (Fix Error)
======================== */
app.get("/", (req, res) => {
  res.send("✅ Microservice is running successfully!");
});

/* ========================
   In-Memory Data
======================== */
let students = [
  { id: 1, name: "Harsha", age: 22 },
  { id: 2, name: "Ravi", age: 21 }
];
```

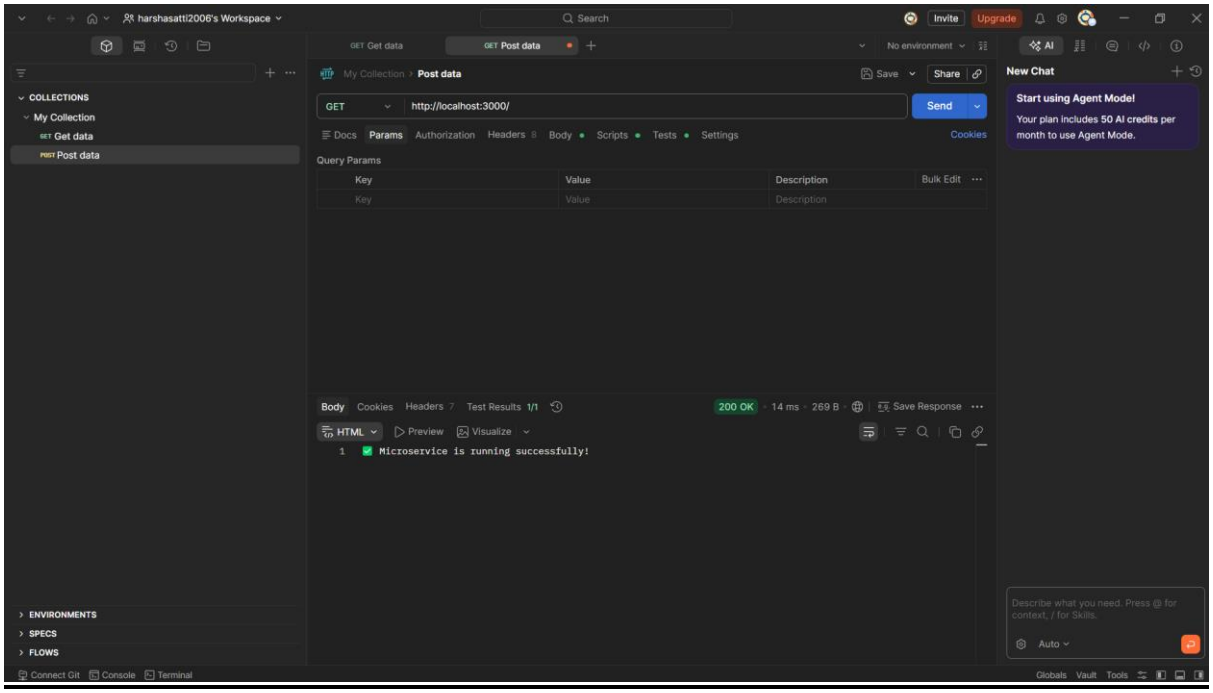
```
/* ========================
   SERVICE LOGIC
========================= */
function getAllStudents() {
  return students;
}

function getStudentById(id) {
  return students.find(s => s.id === id);
}

function addStudent(student) {
  students.push(student);
  return student;
}

function updateStudent(id, data) {
  let student = students.find(s => s.id === id);
  if (!student) return null;

  student.name = data.name || student.name;
  student.age = data.age || student.age;
  return student;
}

function deleteStudent(id) {
  let index = students.findIndex(s => s.id === id);
  if (index === -1) return false;
```

```
  students.splice(index, 1);
  return true;
}

/* ==========================
   REST APIs
========================== */

// GET all students
app.get("/students", (req, res) => {
  res.json(getAllStudents());
});

// GET student by ID
app.get("/students/:id", (req, res) => {
  const student = getStudentById(parseInt(req.params.id));
  if (!student) return res.status(404).json({ error: "Not found" });
  res.json(student);
});

// ADD student
app.post("/students", (req, res) => {
  if (!req.body.id || !req.body.name || !req.body.age) {
    return res.status(400).json({ error: "Invalid input" });
  }
  const student = addStudent(req.body);
  res.status(201).json(student);
});
```

```
// UPDATE student
app.put("/students/:id", (req, res) => {
  const updated = updateStudent(parseInt(req.params.id), req.body);
  if (!updated) return res.status(404).json({ error: "Not found" });
  res.json(updated);
});

// DELETE student
app.delete("/students/:id", (req, res) => {
  const deleted = deleteStudent(parseInt(req.params.id));
  if (!deleted) return res.status(404).json({ error: "Not found" });
  res.json({ message: "Deleted successfully" });
});

/* ========================
   MANUAL UNIT TESTS
======================== */
function runTests() {
  console.log("\nRunning Unit Tests...\n");

  console.log("Test 1: Get All Students");
  console.log(getAllStudents().length > 0 ? "PASS" : "FAIL");

  console.log("Test 2: Get Student By ID");
  console.log(getStudentById(1)?.name === "Harsha" ? "PASS" : "FAIL");

  console.log("Test 3: Add Student");
  let newStudent = addStudent({ id: 3, name: "Kiran", age: 23 });
  console.log(newStudent.name === "Kiran" ? "PASS" : "FAIL");
```

```
  console.log("Test 4: Update Student");
  let updated = updateStudent(1, { name: "Updated" });
  console.log(updated.name === "Updated" ? "PASS" : "FAIL");

  console.log("Test 5: Delete Student");
  let deleted = deleteStudent(2);
  console.log(deleted ? "PASS" : "FAIL");

  console.log("\n✅ All Tests Completed!\n");
}

/* ==========================
   START SERVER
========================== */
const PORT = 3000;

app.listen(PORT, () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`);

  // Run tests automatically
  runTests();
});
```

Outputs:



Result: The microservice was successfully developed and tested.
All unit test cases for service logic and data handling executed correctly and returned expected results.
The REST API endpoints functioned properly for all CRUD operations, confirming the reliability and correctness of the microservice.